Force-Field Optimization via AWH, Frozen-Bias Replay, and Information Geometry

Alexandre Blanco González

April 17, 2026

Contents

1 Overview

This document describes the optimization pipeline implemented in the current repository state. The method alternates between adaptive AWH sampling, frozen-bias readiness probes, frozen-bias production collection, and offline reweighting-based optimization of bounded force-field parameters. The key point is that the optimization gradients are *not* obtained by differentiating the online AWH update rule. Instead, the adaptive AWH phase is used to construct a reliable reference ensemble. Once that reference ensemble has passed readiness checks, the bias is frozen and all thermodynamic estimators are evaluated offline from replayed frames. The idea behind using an extended ensemble as reference is to make the most out of the conformational richness that enhanced sampling simulations provide.

The repository currently supports a general coefficient-weighted thermodynamic cycle. The default example is ethanol hydration, with a solvent leg and a vacuum leg, but the mathematical structure is a sum over arbitrary legs with per-leg lambda schedules, ensembles, and coefficients. The next section develops the formal replay and information-geometric derivations. The implementation section that follows then describes the concrete algorithmic choices used by the code.

2 Frozen-Bias Reference Ensemble

The implemented method separates two tasks:

1. adaptive AWH sampling, which is used to construct a reliable reference extended ensemble, and

2. offline thermodynamic estimation, which is performed only after that reference ensemble has been frozen.

The optimization gradients in the current code are not obtained by differentiating the online AWH update rule. Instead, the adaptive AWH phase is used only to

produce a stationary reference distribution from which replay-based estimators can be evaluated cleanly.

Throughout this section, $i \in \Lambda_\ell$ denotes a generic lambda-state index, $m \in \Lambda_\ell$ denotes a target lambda state to which the stored frames are reweighted, and n indexes stored frames from the frozen reference trajectory. Target-state quantities are indexed by m in subscripts, while m appears as a function argument only when indicating evaluation at that same lambda state.

Consider one thermodynamic leg ℓ with lambda states $i \in \Lambda_\ell$, inverse temperature β_ℓ, and, for NPT legs, pressure P_ℓ. The reduced potential of state i at parameters θ is defined as:

$$u_\ell(\mathbf{x}, V, i; \theta) = \beta_\ell U_\ell(\mathbf{x}, i; \theta) + \beta_\ell P_\ell V, \tag{1}$$

with the pressure-volume term omitted for legs that are not run at constant pressure.

Assume now we have run AWH until reasonable convergence has been achieved. We can then run a frozen-bias AWH simulation, that is, a simulation in which the bias and target lambda weights are no longer being updated, but the system can still jump between lambda-states. The reference Gibbs log-weights used are:

$$g_{\mathrm{ref},\ell}(i) = f_\ell(i) + \log \rho_\ell(i), \tag{2}$$

where $f_\ell(i)$ is the frozen AWH free-energy estimate and $\rho_\ell(i)$ is the fixed target lambda distribution. The corresponding frozen extended-ensemble reference density therefore is:

$$p_{\mathrm{ref},\ell}(\mathbf{x}, V, i) = \frac{\exp\left[g_{\mathrm{ref},\ell}(i) - u_{\mathrm{ref},\ell}(\mathbf{x}, V, i)\right]}{\mathcal{Z}_{\mathrm{ref},\ell}}, \tag{3}$$

where $u_{\mathrm{ref},\ell}$ denotes the reduced potential at the frozen reference parameterization and $\mathcal{Z}_{\mathrm{ref},\ell}$ is the corresponding normalizing constant.

We can obtain the marginal of the conformational-space density by summing over all $i \in \Lambda_\ell$:

$$\begin{aligned} p_{\mathrm{ref},\ell}(\mathbf{x}, V) &= \sum_{i \in \Lambda_\ell} p_{\mathrm{ref},\ell}(\mathbf{x}, V, i) \\ &= \frac{1}{\mathcal{Z}_{\mathrm{ref},\ell}} \sum_{i \in \Lambda_\ell} \exp\left[g_{\mathrm{ref},\ell}(i) - u_{\mathrm{ref},\ell}(\mathbf{x}, V, i)\right] \\ &= \frac{\mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)}{\mathcal{Z}_{\mathrm{ref},\ell}} \end{aligned} \tag{4}$$

Here

$$\mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V) = \sum_{i \in \Lambda_\ell} \exp\left[g_{\mathrm{ref},\ell}(i) - u_{\mathrm{ref},\ell}(\mathbf{x}, V, i)\right]$$

is the pointwise mixture normalizer obtained by summing over lambda states at fixed $(\mathbf{x}, V)$. It is not a global partition function; rather, it is the local normalizing factor that appears when the frozen extended ensemble is marginalized over the lambda coordinate.

3 Replay Reweighting to an Arbitrary Lambda State

Having run an AWH simulation along a λ coordinate allows us to tap into a rich conformational ensemble. For a uniform target ρ, a well-converged AWH bias promotes broad traversal of the lambda schedule and improves state-to-state mixing. Temporal correlations in conformations, volumes, and even lambda visits can remain significant, which is why the implementation later uses split-half checks and support diagnostics rather than assuming independent sampling. In cases where the free energy of interest depends only on terminal lambda states, as for solvation free energies, one need not discard frames generated away from those endpoints: they can still contribute through replay reweighting.

Let $m \in \Lambda_\ell$ be a target lambda state at which we want to evaluate a free energy or a free-energy gradient under the current parameter vector θ. We define the target-state partition function:

$$Q_{\ell,m}(\theta) = \int dV \int d\mathbf{x} \, \exp\left[-u_\ell(\mathbf{x}, V, m; \theta)\right]. \tag{5}$$

By definition, the corresponding reduced free energy is:

$$F_{\ell,m}(\theta) = -\log Q_{\ell,m}(\theta). \tag{6}$$

Equation (4) implies the pointwise identity

$$1 = \frac{\mathcal{Z}_{\mathrm{ref},\ell} p_{\mathrm{ref},\ell}(\mathbf{x}, V)}{\mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)} = \mathcal{Z}_{\mathrm{ref},\ell} p_{\mathrm{ref},\ell}(\mathbf{x}, V) \exp\left[-\log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right]. \tag{7}$$

To connect $Q_{\ell,m}$ to the frozen reference ensemble, we insert Equation (7) into the integrand of Equation (5) and obtain:

$$\begin{aligned} Q_{\ell,m}(\theta) &= \int dV \int d\mathbf{x} \, \mathcal{Z}_{\mathrm{ref},\ell} p_{\mathrm{ref},\ell}(\mathbf{x}, V) \exp\left[-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right] \\ &= \mathcal{Z}_{\mathrm{ref},\ell} \, \mathbb{E}_{\mathrm{ref},\ell}\left[\exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\right]. \end{aligned} \tag{8}$$

Taking minus the logarithm of Equation (8) gives:

$$F_{\ell,m}(\theta) = -\log \mathcal{Z}_{\mathrm{ref},\ell} - \log \mathbb{E}_{\mathrm{ref},\ell}\left[\exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\right]. \quad (9)$$

The first term depends only on the frozen reference ensemble and is therefore a constant shared by every state of the same leg. We do not require that constant explicitly since only free-energy differences are used. It is therefore convenient to define the additive-constant-free quantity:

$$\widetilde{F}_{\ell,m}(\theta) = -\log \mathbb{E}_{\mathrm{ref},\ell}\left[\exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\right], \quad (10)$$

so that

$$F_{\ell,m}(\theta) = -\log \mathcal{Z}_{\mathrm{ref},\ell} + \widetilde{F}_{\ell,m}(\theta). \quad (11)$$

All endpoint differences within a leg depend only on $\widetilde{F}_{\ell,m}(\theta)$ because the reference constant cancels.

4 Discrete Replay Weights Used by the Code

Suppose the frozen reference trajectory for leg ℓ contains M_ℓ stored frames. Replacing the expectation in Equation (10) by the empirical average over those frames gives:

$$\widetilde{F}_{\ell,m}(\theta) \approx -\log\left[\frac{1}{M_\ell}\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta)\right], \quad (12)$$

where the unnormalized target-state replay weights are

$$w^{\mathrm{rep}}_{\ell,n,m}(\theta) = \exp\left[-u_\ell(\mathbf{x}_n, V_n, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}_n, V_n)\right]. \quad (13)$$

Because the prefactor $1/M_\ell$ is the same for every state of the same leg, the implementation evaluates

$$\widetilde{F}_{\ell,m}(\theta) = -\log \sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta), \quad (14)$$

which differs from Equation (12) only by the additive constant $\log M_\ell$ and therefore produces the same endpoint free-energy differences.

The normalized target-state replay weights used to reweight observables to the target state m are

$$\overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) = \frac{w^{\mathrm{rep}}_{\ell,n,m}(\theta)}{\sum_{j=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,j,m}(\theta)}. \tag{15}$$

These are the weights implicitly formed by the current endpoint-gradient routine in the code. They replay each stored configuration frame $(\mathbf{x}_n, V_n)$ at the chosen target lambda state m. In practice, this replay-weight family is the one used whenever the code evaluates endpoint free energies or endpoint free-energy gradients at a chosen target state.

The same machinery can be applied to every lambda state in the schedule to reconstruct a full replayed free-energy profile. Stage B readiness uses this full-profile reconstruction to compare replayed profiles against the frozen AWH profile. In the current implementation, optimization only needs the endpoint states. However, the same replay weights can also be used for more general target-state observables, as derived next.

5 Endpoint Free-Energy Gradients

The endpoint gradient estimator follows directly from the replay expression above. Differentiating Equation (14) with respect to θ gives:

$$\begin{aligned}
\nabla_\theta \widetilde{F}_{\ell,m}(\theta) &= -\frac{1}{\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta)} \sum_{n=1}^{M_\ell} \nabla_\theta w^{\mathrm{rep}}_{\ell,n,m}(\theta) \\
&= -\frac{1}{\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta)} \sum_{n=1}^{M_\ell} \left[-w^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta u_\ell(\mathbf{x}_n, V_n, m; \theta) \right] \\
&= \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta u_\ell(\mathbf{x}_n, V_n, m; \theta).
\end{aligned} \tag{16}$$

Since the pressure-volume term in Equation (1) does not depend on the force-field parameters,

$$\nabla_\theta u_\ell(\mathbf{x}_n, V_n, m; \theta) = \beta_\ell \nabla_\theta U_\ell(\mathbf{x}_n, m; \theta), \tag{17}$$

and therefore

$$\nabla_\theta \widetilde{F}_{\ell,m}(\theta) = \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta)\, \beta_\ell \nabla_\theta U_\ell(\mathbf{x}_n, m; \theta). \tag{18}$$

This is the precise sense in which the replay weights are used to compute endpoint gradients in the implementation: every stored frame is replayed at the

lambda state of interest, and the corresponding force-field gradient at that endpoint is averaged with the normalized target-state replay weights $\overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta)$.

The endpoint free energy of a leg is the coupled-minus-decoupled difference

$$\Delta G_\ell(\theta) = \widetilde{F}_{\ell,\mathrm{coupled}}(\theta) - \widetilde{F}_{\ell,\mathrm{decoupled}}(\theta), \tag{19}$$

and its gradient is

$$\nabla_\theta \Delta G_\ell(\theta) = \nabla_\theta \widetilde{F}_{\ell,\mathrm{coupled}}(\theta) - \nabla_\theta \widetilde{F}_{\ell,\mathrm{decoupled}}(\theta). \tag{20}$$

For a coefficient-weighted thermodynamic cycle,

$$\Delta G_{\mathrm{cyc}}(\theta) = \Delta G_{\mathrm{std}} + \sum_\ell c_\ell \Delta G_\ell(\theta), \tag{21}$$

and the residual against experiment is

$$\mathcal{E}(\theta) = \Delta G_{\mathrm{cyc}}(\theta) - \widetilde{\Delta G}_{\mathrm{exp}}. \tag{22}$$

The code applies a Huber derivative to this residual, so the optimization gradient is actually:

$$\nabla_\theta \mathcal{L} = \frac{d\mathcal{L}}{d\mathcal{E}} \nabla_\theta \Delta G_{\mathrm{cyc}}(\theta). \tag{23}$$

6 General Reweighted Observables

The endpoint free-energy formulas above are not the only quantities that can be estimated from the frozen reference mixture. The same replay weights of Equation (15) also allow us to estimate an arbitrary equilibrium observable of a chosen target lambda state. This is the natural extension needed when one wishes to optimize against targets other than free energies while preserving the same frozen-bias replay framework.

Let $O_{\ell,m}(\mathbf{x}, V; \theta)$ denote a scalar observable evaluated at target lambda state $m \in \Lambda_\ell$. The corresponding target-state equilibrium expectation is

$$\langle O \rangle_{\ell,m}(\theta) = \frac{\int dV \int d\mathbf{x}\, O_{\ell,m}(\mathbf{x}, V; \theta) \exp\left[-u_\ell(\mathbf{x}, V, m; \theta)\right]}{Q_{\ell,m}(\theta)}. \tag{24}$$

To rewrite Equation (24) in terms of the frozen reference ensemble, we use the same pointwise identity of Equation (7). Inserting it into the numerator of Equation (24) gives

$$\int dV \int d\mathbf{x}\, O_{\ell,m}(\mathbf{x}, V; \theta) \exp\left[-u_\ell(\mathbf{x}, V, m; \theta)\right]$$
$$= \int dV \int d\mathbf{x}\, \mathcal{Z}_{\mathrm{ref},\ell} p_{\mathrm{ref},\ell}(\mathbf{x}, V) O_{\ell,m}(\mathbf{x}, V; \theta) \exp\left[-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right]$$
$$= \mathcal{Z}_{\mathrm{ref},\ell} \mathbb{E}_{\mathrm{ref},\ell}\Big[O_{\ell,m}(\mathbf{x}, V; \theta) \exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\Big]. \quad (25)$$

The denominator is exactly the target-state partition function already rewritten in Equation (8),

$$Q_{\ell,m}(\theta) = \mathcal{Z}_{\mathrm{ref},\ell} \mathbb{E}_{\mathrm{ref},\ell}\left[\exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\right]. \quad (26)$$

Substituting Equations (25) and (26) into Equation (24) and cancelling the common factor $\mathcal{Z}_{\mathrm{ref},\ell}$ yields

$$\langle O \rangle_{\ell,m}(\theta) = \frac{\mathbb{E}_{\mathrm{ref},\ell}\left[O_{\ell,m}(\mathbf{x}, V; \theta) \exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\right]}{\mathbb{E}_{\mathrm{ref},\ell}\left[\exp\left(-u_\ell(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)\right)\right]}. \quad (27)$$

To pass from Equation (27) to a practical estimator, we replace the frozen-reference expectations by empirical averages over the stored frames, exactly as in Equation (12). For each stored frame $(\mathbf{x}_n, V_n)$, the Boltzmann factor appearing in both numerator and denominator is the unnormalized target-state replay weight of Equation (13),

$$w^{\mathrm{rep}}_{\ell,n,m}(\theta) = \exp\left[-u_\ell(\mathbf{x}_n, V_n, m; \theta) - \log \mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}_n, V_n)\right].$$

This leads to

$$\langle O \rangle_{\ell,m}(\theta) \approx \frac{\sum_{n=1}^{M_\ell} O_{\ell,m}(\mathbf{x}_n, V_n; \theta) w^{\mathrm{rep}}_{\ell,n,m}(\theta)}{\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta)}. \quad (28)$$

Recognizing the normalized replay weights of Equation (15), we then obtain the discrete replay estimator

$$\widehat{O}_{\ell,m}(\theta) = \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) O_{\ell,m}(\mathbf{x}_n, V_n; \theta), \quad (29)$$

which approximates the exact target-state expectation $\langle O \rangle_{\ell,m}(\theta)$. Equation (29) is the key replay estimator for non-free-energy targets. Any observable that can be evaluated on replayed frames at a chosen state m can, in principle, be optimized in this way.

Gradient of a replayed observable. To differentiate Equation (29), define for brevity

$$O_{\ell,n,m}(\theta) = O_{\ell,m}(\mathbf{x}_n, V_n; \theta)$$

and

$$W_{\ell,m}(\theta) = \sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta).$$

Then Equation (29) may be written as

$$\widehat{O}_{\ell,m}(\theta) = \frac{\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta) O_{\ell,n,m}(\theta)}{W_{\ell,m}(\theta)}. \tag{30}$$

Differentiating Equation (30) by the quotient rule gives

$$\begin{aligned}\nabla_\theta \widehat{O}_{\ell,m}(\theta) = &\frac{\sum_{n=1}^{M_\ell} \left[\nabla_\theta w^{\mathrm{rep}}_{\ell,n,m}(\theta)\, O_{\ell,n,m}(\theta) + w^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta O_{\ell,n,m}(\theta)\right]}{W_{\ell,m}(\theta)} \\ &- \frac{\left[\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta) O_{\ell,n,m}(\theta)\right] \nabla_\theta W_{\ell,m}(\theta)}{W_{\ell,m}(\theta)^2}.\end{aligned} \tag{31}$$

From Equation (13),

$$\nabla_\theta w^{\mathrm{rep}}_{\ell,n,m}(\theta) = -w^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta u_\ell(\mathbf{x}_n, V_n, m; \theta), \tag{32}$$

and therefore

$$\nabla_\theta W_{\ell,m}(\theta) = -\sum_{n=1}^{M_\ell} w^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta u_\ell(\mathbf{x}_n, V_n, m; \theta). \tag{33}$$

Substituting Equations (32) and (33) into Equation (31), and then rewriting the result in terms of the normalized weights of Equation (15), yields

$$\begin{aligned}\nabla_\theta \widehat{O}_{\ell,m}(\theta) = &\sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta O_{\ell,n,m}(\theta) \\ &- \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) \left(O_{\ell,n,m}(\theta) - \widehat{O}_{\ell,m}(\theta)\right) \nabla_\theta u_\ell(\mathbf{x}_n, V_n, m; \theta).\end{aligned} \tag{34}$$

As before, the pressure-volume term does not depend on the force-field parameters, so

$$\nabla_\theta \widehat{O}_{\ell,m}(\theta) = \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) \nabla_\theta O_{\ell,n,m}(\theta) - \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) \Big(O_{\ell,n,m}(\theta) - \widehat{O}_{\ell,m}(\theta) \Big) \beta_\ell \nabla_\theta U_\ell(\mathbf{x}_n, m; \theta). \tag{35}$$

The first term captures any explicit parameter dependence of the observable itself. The second term is the covariance-like reweighting correction arising from the parameter dependence of the Boltzmann factor. If the observable depends only on the stored coordinates and volume, and not directly on θ, then the first term vanishes and one obtains

$$\nabla_\theta \widehat{O}_{\ell,m}(\theta) = - \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{rep}}_{\ell,n,m}(\theta) \Big(O_{\ell,n,m} - \widehat{O}_{\ell,m}(\theta) \Big) \beta_\ell \nabla_\theta U_\ell(\mathbf{x}_n, m; \theta). \tag{36}$$

This is the direct analogue, within the present frozen-AWH setting, of ensemble-reweighting gradient formulas derived for observables in a single-state reference ensemble. The crucial difference here is that the normalized weights are the AWH-aware replay weights of Equation (15), whose denominator uses the frozen Gibbs mixture $\mathcal{Z}_{\mathrm{mix},\ell}(\mathbf{x}, V)$ rather than a single-state reference Boltzmann factor.

Observable-based loss terms. Suppose now that $r \in \mathcal{R}$ indexes one observable target. Each such target is associated with a particular leg and target state, so $\widehat{O}_r(\theta)$ is shorthand for the corresponding replay estimator $\widehat{O}_{\ell_r,m_r}(\theta)$, and $\widetilde{O}_{\mathrm{exp},r}$ denotes its reference value. The corresponding residual is

$$\mathcal{E}_r(\theta) = \widehat{O}_r(\theta) - \widetilde{O}_{\mathrm{exp},r}. \tag{37}$$

If this residual is passed through any differentiable scalar loss function $\mathcal{L}_r(\mathcal{E}_r)$, then its gradient is simply

$$\nabla_\theta \mathcal{L}_r(\mathcal{E}_r(\theta)) = \frac{d\mathcal{L}_r}{d\mathcal{E}_r} \nabla_\theta \widehat{O}_r(\theta). \tag{38}$$

More generally, if the total objective contains several contributions, such as free-energy-cycle terms together with observable-matching terms, one may write

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \sum_{r \in \mathcal{R}} \mathcal{L}_r(\mathcal{E}_r(\theta)), \tag{39}$$

with total gradient

$$\nabla_\theta \mathcal{L}_{\mathrm{tot}}(\theta) = \sum_{r \in \mathcal{R}} \nabla_\theta \mathcal{L}_r(\mathcal{E}_r(\theta)). \tag{40}$$

Equation (23) is therefore the currently implemented special case in which $\mathcal{R}$ contains only the thermodynamic-cycle contribution. Extending the method to new observables does not require changing the frozen-bias replay machinery itself; it only requires adding further target contributions whose predictions and gradients are computed from the same replay weights.

7 Bounded Coordinates

However, the optimizer does not update unconstrained physical parameters directly. Each trainable parameter is represented by a latent coordinate ϕ_i and mapped into a physical parameter through a family-specific transform

$$\theta_i = T_i(\phi_i).$$

For Lennard-Jones σ and ϵ parameters, T_i is a shifted sigmoid that maps into a bounded interval $[\theta_{i,\min}, \theta_{i,\max}]$:

$$\theta_i(\phi_i) = \theta_{i,\min} + (\theta_{i,\max} - \theta_{i,\min})\, \sigma(k\phi_i), \tag{41}$$

where k is a configurable slope and σ is the logistic function. For the current charge-training path, electronegativity coordinates use an affine map while hardness coordinates use a positive softplus-with-floor map. The chain rule then gives, in full generality:

$$\nabla_{\phi_i} U = \frac{d\theta_i}{d\phi_i} \nabla_{\theta_i} U, \tag{42}$$

so all optimization-space gradients and Fisher matrices are assembled in latent coordinates. From this point onward, whenever a reduced potential, free energy, or observable gradient is written as a function of ϕ, this denotes composition with the family-specific map $\theta = T(\phi)$; for example, $u_\ell(\mathbf{x}, V, i; \phi)$ is shorthand for $u_\ell(\mathbf{x}, V, i; \theta(\phi))$. Accordingly, the endpoint replay formulas above are written in the physical parameters θ, whereas the optimizer-side scores, Fisher matrices, and candidate-ensemble reweighting factors below are written in the latent coordinates ϕ. The remaining sections of this theory part establish the formal geometric objects used by the optimizer: the latent-space loss gradient, the candidate-ensemble score, and the Fisher metric induced by the local KL expansion. Practical design choices such as readiness gates, diagonal preconditioning, eigentruncation, confidence moderation, and line-search acceptance are implementation decisions

described later in the pipeline section rather than consequences of the derivations below.

8 From Euclidean to Natural Gradient Descent

At this point the reason for introducing a metric tensor in the optimization step can be stated explicitly: if one ignored the geometry of the probability distributions and performed ordinary Euclidean gradient descent in latent parameter space, the step would be:

$$\Delta\phi_{\mathrm{Euc}} = -\eta\nabla_{\phi}\mathcal{L}. \tag{43}$$

This treats all coordinate directions in ϕ-space as geometrically equivalent. That approximation is usually poor for force-field optimization. Different parameters perturb the sampled distribution with very different strength: a small change in one parameter can produce a large statistical distortion (*e.g.,*, a Lennard-Jones σ)), while a numerically equivalent change in another parameter can have only a modest effect (*e.g.,* a torsion force constant). In other words, the relevant geometry is not the Euclidean geometry of the coordinates themselves, but the geometry of the manifold of probability distributions induced by those coordinates.

The natural gradient is the steepest-descent direction on that statistical manifold. If $\mathcal{I}(\phi)$ denotes the local metric tensor, then the corresponding natural-gradient update is:

$$\Delta\phi_{\mathrm{nat}} = -\eta\mathcal{I}(\phi)^{-1}\nabla_{\phi}\mathcal{L}. \tag{44}$$

In our setting, the appropriate local metric is the Fisher information matrix. We use this matrix as a local description of the manifold curvature, so that the update direction is the local steepest-descent direction consistent with distribution space rather than raw coordinate space.

This is exactly how the current implementation uses the Fisher matrix. The code first builds a latent-space loss gradient, then preconditions it by the empirical Fisher matrix to obtain a natural-gradient-like base step. That step is then stabilized by diagonal normalization, eigentruncation, KL-target scaling, and a line search. The KL derivation below explains why this is the correct local geometry to use.

9 Why Fisher is the Metric Tensor

The Fisher information matrix appears as the first non-zero local approximation to the Kullback-Leibler divergence between nearby candidate ensembles. This is

precisely why the current optimizer uses it as a metric tensor: it measures how large a parameter step is in the space of probability distributions, not in the raw coordinate system of the parameters.

For a fixed frozen reference Gibbs scheme $g_{\mathrm{ref},\ell}$ and current latent parameter vector ϕ, we define the candidate extended-ensemble density:

$$p_\phi(\mathbf{x}, V, i) = \frac{\exp\left[g_{\mathrm{ref},\ell}(i) - u_\ell(\mathbf{x}, V, i; \phi)\right]}{\mathcal{Z}_\ell(\phi)}. \tag{45}$$

Here the dependence on ϕ enters only through the reduced potential, understood through the map $\theta = \theta(\phi)$ introduced above; the frozen AWH Gibbs log-weights are held fixed. We can now compare p_ϕ with a nearby candidate $p_{\phi+\delta\phi}$. The KL divergence from the current ensemble to the proposed ensemble is defined as:

$$D_{\mathrm{KL}}\left(p_\phi \,\|\, p_{\phi+\delta\phi}\right) = \mathbb{E}_\phi\left[\log p_\phi - \log p_{\phi+\delta\phi}\right]. \tag{46}$$

If we now Taylor-expand the log density of the proposed distribution around ϕ, we obtain:

$$\log p_{\phi+\delta\phi} = \log p_\phi + \delta\phi^\top \mathbf{s}_\phi + \frac{1}{2}\delta\phi^\top H_\phi \delta\phi + O(\|\delta\phi\|^3), \tag{47}$$

where

$$\mathbf{s}_\phi = \nabla_\phi \log p_\phi \tag{48}$$

is the score vector and

$$H_\phi = \nabla_\phi^2 \log p_\phi \tag{49}$$

is the Hessian of the log density.

Substituting Equation (47) into Equation (46) yields:

$$D_{\mathrm{KL}}\left(p_\phi \,\|\, p_{\phi+\delta\phi}\right) = -\delta\phi^\top \mathbb{E}_\phi[\mathbf{s}_\phi] - \frac{1}{2}\delta\phi^\top \mathbb{E}_\phi[H_\phi]\delta\phi + O(\|\delta\phi\|^3). \tag{50}$$

Under commutability of differentiation and integration, the score expectation vanishes:

$$\mathbb{E}_\phi[\mathbf{s}_\phi] = \int p_\phi \nabla_\phi \log p_\phi = \int \nabla_\phi p_\phi = \nabla_\phi \int p_\phi = \nabla_\phi 1 = 0. \tag{51}$$

Therefore, the linear term disappears and the *first nonzero* contribution to the local KL divergence is second order:

$$D_{\mathrm{KL}}\left(p_\phi \,\|\, p_{\phi+\delta\phi}\right) = \frac{1}{2}\delta\phi^\top \mathcal{I}(\phi)\delta\phi + O(\|\delta\phi\|^3), \tag{52}$$

where

$$\mathcal{I}(\phi) = -\mathbb{E}_\phi[H_\phi] \tag{53}$$

is the Fisher information matrix, and Equation (52) is the reason the Fisher matrix is the natural metric tensor: its quadratic form is the local statistical distance induced by KL divergence.

10 From the KL Expansion to a Covariance of Energy Gradients

Under the standard Fisher regularity conditions, these are; that the support of the probability density does not depend on ϕ, the parameter space is open, the log-density is twice continuously differentiable with respect to ϕ, and differentiation may be interchanged with integration, one obtains the score-covariance identity:

$$\mathcal{I}(\phi) = \mathbb{E}_\phi \left[\mathbf{s}_\phi \mathbf{s}_\phi^\top \right]. \tag{54}$$

To turn this abstract definition into the object actually used by the code, we explicitly derive the expression for the score of the frozen Gibbs bias in Equation (45). Taking the logarithm of that same Equation gives:

$$\log p_\phi(\mathbf{x}, V, i) = g_{\mathrm{ref},\ell}(i) - u_\ell(\mathbf{x}, V, i; \phi) - \log \mathcal{Z}_\ell(\phi). \tag{55}$$

then, differentiating it with respect to ϕ yields:

$$\begin{aligned} \nabla_\phi \log p_\phi(\mathbf{x}, V, i) &= \mathbf{s}_\phi(\mathbf{x}, V, i) \\ &= -\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) - \nabla_\phi \log \mathcal{Z}_\ell(\phi). \end{aligned} \tag{56}$$

Now,

$$\begin{aligned} \nabla_\phi \log \mathcal{Z}_\ell(\phi) &= \frac{1}{\mathcal{Z}_\ell(\phi)} \nabla_\phi \mathcal{Z}_\ell(\phi) \\ &= \frac{1}{\mathcal{Z}_\ell(\phi)} \sum_i \int dV \int d\mathbf{x} \left[-\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) \right] \\ &\qquad \times \exp\left[g_{\mathrm{ref},\ell}(i) - u_\ell(\mathbf{x}, V, i; \phi) \right] \\ &= -\mathbb{E}_\phi \left[\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) \right]. \end{aligned} \tag{57}$$

and substituting Equation (57) into Equation (56) gives:

$$\mathbf{s}_\phi(\mathbf{x}, V, i) = -\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) + \mathbb{E}_\phi \left[\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) \right]. \tag{58}$$

This is already a centered quantity, so at the exact distribution level the Fisher matrix is the covariance of reduced-potential gradients:

$$\mathcal{I}(\phi) = \mathrm{Cov}_\phi \left(\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) \right). \tag{59}$$

Finally, because $u_\ell(\mathbf{x}, V, i; \phi) = \beta_\ell U_\ell(\mathbf{x}, i; \phi) + \beta_\ell P_\ell V$ and the $P_\ell V$ term does not depend on ϕ,

$$\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) = \beta_\ell \nabla_\phi U_\ell(\mathbf{x}, i; \phi), \tag{60}$$

which leads to the form used conceptually by the implementation:

$$\mathcal{I}(\phi) = \mathrm{Cov}_\phi \left(\beta_\ell \nabla_\phi U_\ell(\mathbf{x}, i; \phi) \right). \tag{61}$$

Thus, the Fisher matrix is not an arbitrary preconditioner. It is the covariance of the reduced-potential score vectors and the local metric tensor induced by KL divergence.

11 Empirical Reweighting, ESS, and the Practical Fisher Estimate

The formulas above are expectations under the current candidate distribution p_ϕ, but the stored frames come from the frozen reference distribution $p_{\mathrm{ref},\ell}$. The code therefore reweights the reference frames into the current candidate ensemble. These weights are distinct from the target-state replay weights introduced earlier. There, each stored configuration frame $(\mathbf{x}_n, V_n)$ is replayed at an arbitrarily chosen target lambda state m. Here, instead, each stored extended-state sample $(\mathbf{x}_n, V_n, i_n)$ is reweighted into the current candidate extended ensemble at its physically visited lambda state i_n, so the weights carry no target-state index m and are functions of the latent coordinates ϕ.

For the full extended state $(\mathbf{x}, V, i)$,

$$\frac{p_\phi(\mathbf{x}, V, i)}{p_{\mathrm{ref},\ell}(\mathbf{x}, V, i)} = \frac{\mathcal{Z}_{\mathrm{ref},\ell}}{\mathcal{Z}_\ell(\phi)} \exp\left[-u_\ell(\mathbf{x}, V, i; \phi) + u_{\mathrm{ref},\ell}(\mathbf{x}, V, i) \right]. \tag{62}$$

Because the frozen Gibbs bias $g_{\mathrm{ref},\ell}(i_n)$ appears in both numerator and denominator, it cancels exactly. For a stored frame n with visited lambda state i_n, the unnormalized candidate-ensemble reweighting factor used by the optimizer is therefore:

$$w^{\mathrm{ens}}_{\ell,n}(\phi) \propto \exp\left[-u_\ell(\mathbf{x}_n, V_n, i_n; \phi) + u_{\mathrm{ref},\ell}(\mathbf{x}_n, V_n, i_n) \right]. \tag{63}$$

For NPT legs, the $P_\ell V$ term also cancels because the same pressure is used in the reference and candidate ensembles. The normalized candidate-ensemble weights therefore are:

$$\overline{w}^{\mathrm{ens}}_{\ell,n}(\phi) = \frac{w^{\mathrm{ens}}_{\ell,n}(\phi)}{\sum_{j=1}^{M_\ell} w^{\mathrm{ens}}_{\ell,j}(\phi)}, \tag{64}$$

These normalized candidate-ensemble weights are used for optimizer-side quantities such as the effective sample size and the empirical Fisher estimate; they are not the target-state replay weights used for endpoint free energies, replayed observables, or their gradients. A standard importance-sampling support diagnostic is the Kish-type effective sample size:

$$\mathrm{ESS}_\ell(\phi) = \frac{1}{\sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{ens}}_{\ell,n}(\phi)^2}. \tag{65}$$

which quantifies weight degeneracy under replay into the candidate ensemble. It should be interpreted as a support diagnostic for the importance weights, not as a literal count of statistically independent trajectory samples.

Unlike the endpoint replay gradients above, which use $\nabla_\theta U_\ell(\mathbf{x}_n, m; \theta)$ evaluated at a chosen target state m, the quantity below is the samplewise realization entering the score covariance in latent coordinates and is evaluated at the physically visited state i_n. We calculate the empirical latent-space score vector for frame n at the physically visited lambda state as:

$$\mathbf{s}_{\ell,n}(\phi) = \beta_\ell \nabla_\phi U_\ell(\mathbf{x}_n, i_n; \phi), \tag{66}$$

and the weighted empirical Fisher matrix becomes:

$$\widehat{\mathcal{I}}_\ell(\phi) = \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{ens}}_{\ell,n}(\phi) \left(\mathbf{s}_{\ell,n}(\phi) - \bar{\mathbf{s}}_\ell(\phi)\right) \left(\mathbf{s}_{\ell,n}(\phi) - \bar{\mathbf{s}}_\ell(\phi)\right)^\top, \tag{67}$$

where

$$\bar{\mathbf{s}}_\ell(\phi) = \sum_{n=1}^{M_\ell} \overline{w}^{\mathrm{ens}}_{\ell,n}(\phi) \mathbf{s}_{\ell,n}(\phi). \tag{68}$$

When the total objective is assembled from contributions evaluated on one or more separate frozen ensembles, the corresponding empirical Fisher used by the implementation remains the sum of the leg contributions in the shared latent parameter space. Adding observable targets therefore changes the loss gradient, but not the basic score-covariance construction of the metric tensor.

Because the first nonzero KL term is $\frac{1}{2}\delta\phi^\top \mathcal{I}(\phi)\delta\phi$, the optimizer uses the empirical Fisher matrix to define a trust region,

$$D_{\mathrm{KL}} \approx \frac{1}{2}\Delta\phi^\top \widehat{\mathcal{I}}(\phi)\Delta\phi. \tag{69}$$

The empirical Fisher matrix is therefore the operator that turns the Euclidean loss gradient into a natural-gradient-like base step, after which the optimizer applies diagonal normalization, eigentruncation, and KL-target scaling before the line search. The full logic is:

1. replay the frozen reference frames to obtain the target predictions and their gradients,
2. assemble the latent-space loss gradient from those target contributions,
3. estimate the empirical Fisher matrix from reweighted score vectors at the visited lambda states,
4. use that Fisher matrix as the local metric tensor of the statistical manifold,
5. bound the proposed update through the quadratic KL model in Equation (69).

This is why a KL target, rather than a raw Euclidean step length, is the natural quantity to bound. A step that is small in Euclidean coordinates can still be large in distribution space if it distorts the ensemble strongly, while a step that is large in a weakly constrained coordinate may still be statistically safe. The Fisher metric captures that distinction directly.

The derivations in this section establish the replay estimators and the local KL geometry used by the optimizer. They do not, by themselves, fix pragmatic choices such as Stage B readiness gates, diagonal preconditioning, eigentruncation thresholds, confidence moderation, or line-search acceptance logic. Those are algorithmic design choices of the current implementation. The next section summarizes the current postprocessing and step-control pipeline used to turn the empirical Fisher estimate into a robust update.

12 Numerical Conditioning and Step Control

The raw empirical Fisher matrix $\widehat{\mathcal{I}}(\phi)$ is often ill-conditioned. This is a practical consequence of finite sampling: some parameter directions, or linear combinations thereof, may only weakly affect the sampled ensemble, so the corresponding eigenvalues are very small and the formal natural-gradient step becomes numerically

unstable. The role of this section is therefore not to introduce a new geometric identity, but to describe how the implementation converts the empirical Fisher estimate into a robust update. The current code always applies Jacobi preconditioning, eigentruncation, and KL-target scaling, and it can additionally apply diagonal regularization through an optional reference-model penalty.

12.1 Diagonal Regularization

When a nonzero reference-penalty strength is configured for a parameter pool, the implementation adds a quadratic reference-model penalty to the loss to discourage the corresponding parameters from drifting too far from their starting values. For parameter p with strength λ_p, the penalty is $\frac{\lambda_p}{2}(\theta_p - \theta_{\mathrm{ref},p})^2$. In latent coordinates, this penalty contributes two simultaneous terms. Its gradient with respect to ϕ_p is

$$\lambda_p \left(\theta_p(\phi_p) - \theta_{\mathrm{ref},p}\right) \frac{d\theta_p}{d\phi_p}, \tag{70}$$

which is added directly to the latent-space loss gradient $\nabla_\phi \mathcal{L}$, actively pushing the optimizer back toward the reference parameterization. Its second derivative contributes a diagonal curvature term,

$$\widehat{\mathcal{I}}_{pp} \leftarrow \widehat{\mathcal{I}}_{pp} + \lambda_p \left(\frac{d\theta_p}{d\phi_p}\right)^2, \tag{71}$$

which is added to the empirical Fisher diagonal before preconditioning, lifting poorly informed diagonal directions and reducing the risk of unstable steps. Both contributions are present simultaneously: the gradient term penalises the current deviation from the reference, while the curvature term encodes the stiffness of that penalty in the local metric. If the added diagonal contribution is strictly positive in all retained directions, the regularized matrix becomes strictly positive definite; more generally, it still improves conditioning even when that stronger statement does not hold globally.

12.2 Jacobi Preconditioning

Before eigentruncation, the code applies a diagonal normalization, also known as Jacobi preconditioning. Its main purpose is to make the truncation criterion insensitive to the arbitrary physical scales and units of the force-field parameters (e.g., kJ/mol vs. nm vs. elementary charge). Let D be the diagonal matrix of the empirical Fisher matrix, $D = \mathrm{diag}(\widehat{\mathcal{I}})$. The preconditioned Fisher matrix and scaled gradient are formed as:

$$\widehat{\mathcal{I}}_{\text{corr}} = D^{-1/2}\widehat{\mathcal{I}}D^{-1/2}, \quad \mathbf{g}_{\text{scaled}} = D^{-1/2}\nabla_{\phi}\mathcal{L}. \tag{72}$$

In the limit of exact inversion, this diagonal rescaling does not change the final natural-gradient step, because the factors of D cancel out when the matrix is inverted and mapped back to the original coordinates. Its practical importance appears once eigentruncation is introduced: after rescaling, the retained and discarded directions are determined by the correlation structure of the empirical Fisher matrix rather than by the raw magnitudes induced by parameter units.

Without Jacobi preconditioning, eigentruncation could discard physically relevant directions simply because those coordinates have small numerical curvature in their native units. After rescaling, each coordinate is measured relative to its own local diagonal curvature, making $\widehat{\mathcal{I}}_{\text{corr}}$ behave like a correlation-like matrix. The subsequent truncation then preferentially removes directions that are weakly informed by the sampled reference ensemble, rather than directions that merely happen to be expressed in small numerical units.

12.3 Eigentruncation

The preconditioned matrix $\widehat{\mathcal{I}}_{\text{corr}}$ is then decomposed into its eigenvalues λ_j and eigenvectors $\mathbf{v}_j$. Directions corresponding to eigenvalues below a relative tolerance ϵ_{tol} (with the current default equal to 10^{-3} of the maximum eigenvalue) are treated as weakly informed or nearly redundant and are truncated. The regularized inverse is constructed as:

$$\widehat{\mathcal{I}}^{\dagger}_{\text{corr}} = \sum_{\lambda_j > \epsilon_{\text{tol}}\lambda_{\max}} \frac{1}{\lambda_j}\mathbf{v}_j\mathbf{v}_j^{\top}. \tag{73}$$

This eigentruncation prevents the optimizer from taking very large steps along directions for which the empirical Fisher provides little reliable curvature information.

12.4 KL-Target Scaling and Confidence Moderation

The base latent-space step is then computed as:

$$\Delta\phi_{\text{base}} = D^{-1/2}\widehat{\mathcal{I}}^{\dagger}_{\text{corr}}\mathbf{g}_{\text{scaled}}. \tag{74}$$

The optimizer next imposes a trust-region bound in distribution space. Because the first nonzero term in the local KL expansion is $\frac{1}{2}\delta\phi^{\top}\mathcal{I}(\phi)\delta\phi$, the code uses the quadratic KL model from Equation (69) to rescale the base step so that the predicted divergence does not exceed a target value δ_{KL}:

$$\Delta\phi = \alpha_{\mathrm{KL}}\,\Delta\phi_{\mathrm{base}}, \quad \text{with } \alpha_{\mathrm{KL}} = \min\left(1, \sqrt{\frac{2\delta_{\mathrm{KL}}}{\Delta\phi_{\mathrm{base}}^{\top}\widehat{\mathcal{I}}\,\Delta\phi_{\mathrm{base}}}}\right). \tag{75}$$

The notation α_{KL} distinguishes this static rescaling from the separate halving factor $\alpha \in \{1, \frac{1}{2}, \frac{1}{4}, \ldots\}$ used by the backtracking line search described in the implementation section. Both are applied in sequence: $\Delta\phi_{\mathrm{base}}$ is first rescaled by α_{KL} to produce a KL-bounded base step, and the line search then multiplies by α until residual and support conditions are satisfied.

The base KL target δ_{KL} also determines the minimum acceptable effective sample size checked on each line-search step. A parameter change with predicted KL divergence equal to δ_{KL} reduces the Kish ESS of the reference ensemble by a factor of approximately $\exp(-2\delta_{\mathrm{KL}})$. The per-leg ESS threshold is therefore set to

$$\mathrm{ESS}_{\min,\ell} = s_{\mathrm{ESS}} \cdot M_{\ell} \cdot \exp(-2\delta_{\mathrm{KL}}), \tag{76}$$

where $s_{\mathrm{ESS}} \in (0, 1]$ is a configurable safety margin. In the current implementation, this ESS threshold is computed once from the base KL target and is not rescaled by the later confidence factor. The ESS is evaluated at the *proposed* parameters on each halving step, so the line search rejects any proposal that would leave the reference ensemble with insufficient support for the candidate ensemble, independently of whether the residual has improved.

The target δ_{KL} is itself moderated by a confidence factor $s_{\mathrm{conf}} \in [s_{\min}, 1]$. In the implementation, this factor is obtained by comparing optimization-relevant quantities estimated from the first and second halves of the production data, such as target predictions and their gradients. Large split-half disagreement is interpreted as evidence that the current replay estimates are noisy or insufficiently converged, in which case s_{conf} is reduced. This shrinks the trust region and makes the update more conservative. This confidence moderation is therefore an additional robustness heuristic layered on top of the KL trust region, not a separate geometric identity.

The empirical Fisher matrix is therefore the operator that turns the Euclidean loss gradient into a natural-gradient-like base step, after which the optimizer applies the stabilization steps described above. The full logic is:

1. replay the frozen reference frames to obtain the target predictions and their gradients,

2. assemble the latent-space loss gradient from those target contributions,

3. estimate the empirical Fisher matrix from reweighted score vectors at the visited lambda states,

4. use that Fisher matrix as the local metric tensor of the statistical manifold,

5. stabilize and bound the proposed update through diagonal regularization, scale-invariant truncation, and the quadratic KL model in Equation (69).

This is why a KL target, rather than a raw Euclidean step length, is the natural quantity to bound. A step that is small in Euclidean coordinates can still be large in distribution space if it distorts the ensemble strongly, while a step that is large in a weakly constrained coordinate may still be statistically safe. The Fisher metric captures that distinction directly. In that sense, the replay machinery determines which direction lowers the objective, whereas the empirical Fisher determines how far the optimizer can move in that direction before the induced ensemble changes too much.

13 Implemented Pipeline

This section describes the concrete algorithmic design used by the current code. Unlike the theory section, which derives replay estimators and the local Fisher geometry, the present section focuses on operational choices: readiness checks, probe growth policy, production collection, numerical stabilization, and acceptance logic.

The current code organizes the method as a macro-epoch loop. At the beginning of each macro epoch, the pipeline constructs one AWH simulation per thermodynamic leg with the current physical parameters. Restart coordinates and velocities may be reused from the previous macro epoch. The AWH bias is reused only when restart continuity is preserved; a cold restart from the input structure does not reuse a previous bias profile. For each leg, the macro epoch proceeds through four stages:

1. optional rewarm of a reused simulation state,

2. readiness-gated AWH sampling,

3. frozen-bias production collection,

4. offline optimization from the frozen production artifacts.

The optimizer never consumes data from an actively adapting AWH bias.

13.1 Stage A: Cheap Readiness Screening

Stage A is a low-cost check applied after each AWH block. It does not attempt to estimate free energies directly. Instead, it asks whether the adaptive AWH run is mixed and statistically mature enough to justify a frozen-bias probe. The current implementation monitors:

- recent mean bias-change magnitude,
- lambda-history effective sample size,
- Molly's linear-stage effective sample count N_{eff},
- full round trips between the coupled and decoupled states,
- recent endpoint occupancy,
- solvent-tail occupancy floors for late Lennard-Jones states in the solvent leg.

Only after a leg passes these Stage A criteria for a required stable streak does the pipeline launch a Stage B probe. Repeated Stage B failures can increase the required stable streak or introduce cooldown periods before the next probe.

13.2 Stage B: Frozen-Bias Probe and Parity Validation

Stage B is the decisive readiness check. The current leg is cloned into a frozen reference simulation, the AWH bias is held fixed, bias-update accumulators are cleared, and a short probe trajectory is generated. The probe frames are then processed offline:

1. discard an initial fraction of the probe,
2. apply stride and minimum/maximum retained-frame rules,
3. rebuild CPU replay templates for all lambda states,
4. evaluate each retained frame under every state Hamiltonian.

The resulting energy matrix is used to compute two independent Stage B diagnostics.

Split-half convergence. The retained frames are split into temporal halves. A coupled-minus-decoupled free energy is computed independently from each half, and the split gap is the absolute difference between those two estimates.

Replay/AWH parity. The same retained frames are used to reconstruct a full lambda free-energy profile from the frozen reference mixture. That replayed profile is aligned to the coupled endpoint and compared against the frozen AWH profile that generated the frames. The implementation tracks:

- the raw parity gap over all states,
- the supported parity gap over states whose reweighting ESS exceeds a configured support threshold,
- a separate endpoint parity gap,
- minimum support coverage over the full lambda schedule.

The default gate is therefore *support-aware* but not support-blind: a leg must satisfy split convergence, supported parity, endpoint parity, and support coverage simultaneously.

Split-only continuation. The current code distinguishes true split-only failures from genuine parity failures. If split convergence fails while supported parity, endpoint parity, and support coverage all still pass, the pipeline does *not* return immediately to adaptive AWH sampling. Instead, it continues the same frozen probe in place, appending one more segment under the unchanged frozen bias. The raw logger history is concatenated and the entire frame-selection procedure is rerun on the combined probe so that discard, stride, and frame-cap rules remain consistent. This continuation ends as soon as the failure mode is no longer split-only or the configured segment budget is exhausted.

Retry policy. When Stage B fails for lack of support or split convergence, the next probe is grown. When it fails on parity, the probe length is held fixed and the leg is sent back to Stage A so the adaptive bias can continue to improve. Near-pass behavior is leg-specific: by default, the solvent leg keeps the same probe length on a near-pass rather than shrinking it, while the vacuum leg may still shrink.

13.3 Frozen-Bias Production Artifacts

Once every leg in the cycle has passed Stage B, the pipeline launches a second frozen-bias simulation for production collection. For each leg, the current code stores:

- the logged coordinates, active lambda history, and, where relevant, volume history,

- the frozen bias metadata,
- precomputed replay neighbor data,
- an ensemble-evaluation cache for CPU replay,
- the reference energy matrix $u_{\mathrm{ref},\ell}$ over all store frames and all lambda states.

These production artifacts are the only data used by the optimizer.

13.4 Offline Optimization and Step Acceptance

The optimizer evaluates the current candidate parameter vector on the frozen production artifacts and assembles three objects:

1. the coefficient-weighted cycle free energy,
2. the coefficient-weighted cycle gradient in latent space,
3. the empirical Fisher matrix in latent space.

The per-frame force-field gradients $\nabla_\theta U_\ell(\mathbf{x}_n, i; \theta)$ required for both the cycle gradient and the empirical Fisher estimate are obtained by differentiating the Molly potential-energy function with respect to the parameter vector via reverse-mode automatic differentiation, using Enzyme. The chain-rule factor $d\theta_p/d\phi_p$ from the family-specific latent-to-physical transform is then applied to convert these θ-space gradients into latent-space gradients before the Fisher and cycle-gradient assembly.

The base step is formed from the replay gradient and empirical Fisher matrix derived in the theory section, then stabilized through diagonal preconditioning, eigentruncation, KL-target scaling, and a per-pool infinity-norm cap on the latent update. The per-pool cap bounds the maximum absolute latent-coordinate change within each parameter pool independently, using a configurable `max_phi_step` per pool; this allows, for example, a tighter cap on Lennard-Jones σ parameters than on partial-charge parameters. These are practical numerical design choices rather than additional formal identities. The line search then proposes

$$\phi_{\mathrm{prop}} = \phi_{\mathrm{current}} - \alpha\,\Delta\phi, \tag{77}$$

starting from $\alpha = 1$ and halving up to a fixed maximum of seven iterations until three conditions hold simultaneously:

- the effective sample size, evaluated at the *proposed* parameters ϕ_{prop}, remains above the per-leg threshold for every leg,

- the per-pool physical parameter drift relative to the reference model remains within configured bounds for every pool,
- the residual satisfies the acceptance test described below.

The ESS is recomputed at ϕ_{prop} on each halving step, not at the current parameters; it therefore checks whether the frozen reference ensemble still provides adequate importance-sampling support for the proposed candidate ensemble. The drift check is applied to the mapped physical parameters $\theta_{\text{prop}} = \theta(\phi_{\text{prop}})$ and gates per-pool families independently.

The residual test is not a simple monotonic decrease. Let τ be a configurable relative noise tolerance and let $r_{\text{add}} \geq 0$ be the additional improvement requirement from split-half confidence (defined below). A proposal is accepted when

$$|\mathcal{E}_{\text{prop}}| \leq \max(\varepsilon_{\text{floor}},\ |\mathcal{E}| + \tau\,|\mathcal{E}| - r_{\text{add}})\,, \tag{78}$$

where $\mathcal{E}$ is the residual at the current parameters and $\varepsilon_{\text{floor}}$ is a small positive floor derived from the noise tolerance. The window therefore widens by a fraction τ of the current residual magnitude, while r_{add} tightens it when production-data confidence is poor; the floor prevents the acceptance threshold from collapsing to zero. If all seven halvings are exhausted without satisfying all three conditions the line search is declared failed, which triggers an early exit from the inner optimization loop.

Split-half confidence scaling. The production artifacts are also split deterministically into first and second temporal halves. The code recomputes endpoint free energies and gradients on each half and forms three disagreement measures:

- the maximum per-leg endpoint ΔG disagreement,
- the disagreement in the full cycle contribution,
- the disagreement in the cycle gradient vector.

These are compressed into a confidence factor

$$s_{\text{conf}} = \max\left(s_{\min}, \frac{1}{1 + \kappa\,\delta_{\max}}\right), \tag{79}$$

where $\delta_{\max}$ is the largest disagreement signal. The same split-half analysis also adds an extra residual-improvement requirement proportional to the cycle disagreement before a line-search step is accepted.

Inner-loop exits. If the accepted line-search step becomes too small, if the line search repeatedly hits a tiny α, or if the configured inner-epoch cap is reached, the code restores the best state seen inside that macro epoch and returns to fresh resimulation. A fresh macro-epoch Stage B failure does not trigger a hard rejection of the previously accepted parameter set; it simply prevents additional optimization until new readiness is established.

13.5 Charge-Equilibration Parameterization

When charge training is enabled, the code uses a constrained charge-equilibration model as a model-specific parameterization layer on top of the general replay-and-optimization machinery. Rather than optimizing partial charges q_i directly, the trainable physical parameters are the electronegativity χ_i and hardness η_i associated with each atom type or class.

For a molecule with target net charge Q_{mol}, the partial charge on trainable atom i is written as

$$q_i = -\frac{\chi_i + \lambda_{\mathrm{mol}}}{\eta_i}, \tag{80}$$

where λ_{mol} is the molecule-specific Lagrange multiplier that enforces the net-charge constraint. Requiring

$$\sum_{i \in \mathrm{trainable}} q_i = Q_{\mathrm{mol}} - \sum_{j \in \mathrm{fixed}} q_j, \tag{81}$$

yields the analytic solution

$$\lambda_{\mathrm{mol}} = -\frac{Q_{\mathrm{mol}} - \sum_{j \in \mathrm{fixed}} q_j + \sum_{i \in \mathrm{trainable}} \frac{\chi_i}{\eta_i}}{\sum_{i \in \mathrm{trainable}} \frac{1}{\eta_i}}. \tag{82}$$

This construction enforces the molecular net charge exactly by construction while still allowing the optimizer to work in the bounded latent coordinates described earlier. In practice the hardness variables are kept positive through the chosen latent-to-physical transform so that the charge solve remains well conditioned, whereas electronegativity coordinates may be lightly bounded or left effectively unbounded depending on the training setup.

13.6 Current Scope and Default Example

The repository now supports a general thermodynamic cycle with arbitrary legs, coefficients, per-leg lambda schedules, and per-leg ensembles. The default example remains ethanol hydration. In that default setup:

- the solvent leg uses an NPT expanded ensemble with a staged, charge-first and Lennard-Jones-second lambda schedule concentrated in the low-Lennard-Jones tail,
- the vacuum leg uses a simpler non-periodic leg with no pressure-volume term,
- the optimization can jointly train Lennard-Jones parameters and partial charges through the constrained charge-equilibration parameterization described above,
- optimization is solute-only unless solvent parameters are explicitly enabled.

This is therefore no longer a speculative milestone roadmap. The implemented method is a readiness-gated, frozen-bias replay pipeline with support-aware Stage B validation, bounded latent-coordinate optimization, KL-limited natural-gradient steps, and deterministic confidence moderation from split-half production artifacts.